

# Memoria-Practicas-STR.pdf



tomjunior



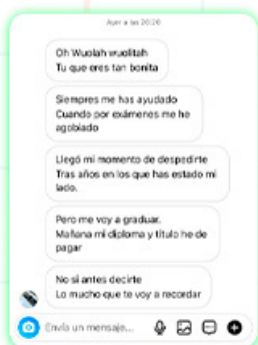
Sistemas en Tiempo Real



4º Grado en Ingeniería Informática



Escuela Politécnica Superior de Córdoba  
Universidad de Córdoba



**Que no te escriban poemas de amor  
cuando terminen la carrera**



*(a nosotros por  
suerte nos pasa)*

**WUOLAH**

# La escuela de Ciberseguridad más grande del mundo.

La formación más completa y transversal que demanda el mercado.

Sabemos que es difícil definir tu futuro profesional  
¿Te ayudamos?

SISTEMAS EN TIEMPO REAL - INGENIERÍA INFORMÁTICA

## MEMORIA DE PRÁCTICAS ROBOT LEGO MINDSTORMS



Trabajo realizado por:

Tomás de la Yglesia Bocero

Javier Gómez Palmero

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

IMF  
Smart Education



**Deloitte.**

Máster en  
Ciberseguridad

Más info



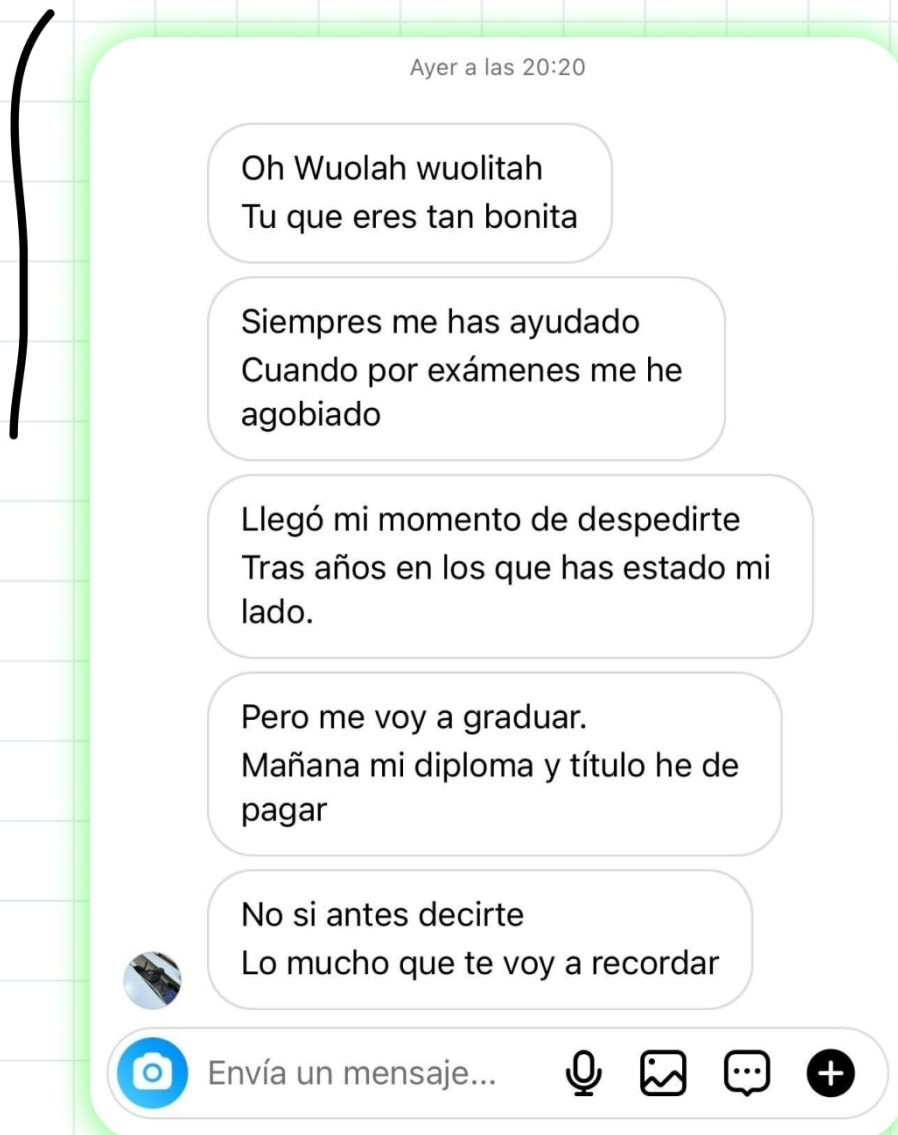
WUOLAH

---

# ÍNDICE

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| <b>ÍNDICE</b>                                                                       | <b>2</b>  |
| <b>Introducción</b>                                                                 | <b>3</b>  |
| ¿Qué es Lego Mindstorms NXT?                                                        | 3         |
| Proceso de Montaje                                                                  | 3         |
| Instalación y software                                                              | 3         |
| <b>Práctica 1.- Manejo básico de Lego Mindstorms NXT: Sensores y Actuadores</b>     | <b>4</b>  |
| Apartado a                                                                          | 4         |
| Apartado b                                                                          | 10        |
| Apartado c                                                                          | 16        |
| <b>Práctica 2.- Manejo básico de Lego Mindstorms NXT: Tareas</b>                    | <b>24</b> |
| <b>Práctica 3.- Manejo avanzado de Lego Mindstorms NXT: Tareas y Comunicaciones</b> | <b>31</b> |

**Que no te escriban poemas de amor  
cuando terminen la carrera ▶▶▶▶▶▶▶▶**  
(a nosotros por suerte nos pasa) 😊



**WUOLAH**

---

## Introducción

### ¿Qué es Lego Mindstorms NXT?

**LEGO Mindstorms NXT** es un set programable fabricado por LEGO en julio de 2006. Este set reemplazó a la primera versión de Lego Mindstorms RCX. Dicho aparato tiene dos versiones, la versión comercial (set#8527) y la versión educativa (set #9797). El set viene con el programa NXT-G u opcionalmente con el programa de LabVIEW para Lego Mindstorms. Una nueva versión del NXT salió en 2009, llamada Lego Mindstorms NXT 2.0, la cual incluía un sensor de color y algunas otras mejoras. Más tarde salió un nuevo modelo llamado EV3, el último a la fecha (2018).

### Proceso de Montaje

Utilizando la guía de montaje de Lego se construyó el robot. Se realizó la estructura añadiendo varios sensores.

### Instalación y software

Para poder empezar a programar el robot primero se tuvo que crear un entorno donde programarlo. El profesor entregó una máquina virtual Windows 7 ya configurada para poder programarlo. Únicamente se necesitó instalar unos drivers en la máquina anfitriona y en la máquina virtual.

Estas prácticas se podían realizar en C o en C++. En este caso se programaron en lenguaje C. Cada práctica estaba formada por los archivos con extensión .c, .oil y el archivo makefile.

Como terminal se usó el programa Cygwin, el cual había que ejecutar como administrador para su debido funcionamiento.

---

# Práctica 1.- Manejo básico de Lego Mindstorms NXT: Sensores y Actuadores

## Apartado a

Calibrar la potencia relativa de los motores para realizar un movimiento lineal con un error menor a 1cm. de desvío por cada 1 m. de avance lineal. (Error menor al 1%).

### Fichero.c

En primer lugar se declaran las tareas *Avance*, *Calibra* y *Final*. Además, se declara un contador y cuatro variables. Estas variables son las siguientes:

- **b**: tipo int, inicializada a 100. Utilizada para la velocidad del motor conectado al puerto b.
- **c**: tipo int, inicializada a 100. Utilizada para la velocidad del motor conectado al puerto c.
- **rpmb**: tipo int, sin inicializar. Utilizada para contar la vueltas del motor conectado al puerto b.
- **rpmc**: tipo int, sin inicializar. Utilizada para contar la vueltas del motor conectado al puerto c.

```
1 #include "kernel.h"
2 #include "kernel_id.h"
3 #include "ecrobot_interface.h"
4
5 /*-----
6  /* OSEK declarations
7  /*-----
8  DeclareCounter(Contador);
9  DeclareTask(Avance);
10 DeclareTask(Calibra);
11 DeclareTask(Final);
12 int b=100, c=100;
13 int rpmb, rpmc;
14
15 /*-----
16 /* Function to be invoked from a category 2 interrupt
17 /*-----
18 void user_lms_isr_type2(){
19     SignalCounter(Contador);
20 }
```



Que no te escriban poemas de amor  
cuando terminen la carrera ▶▶▶▶▶▶▶▶



WUOLAH

(a nosotros por suerte nos pasa)

No si antes decirte  
Mañana mi diploma y título he de  
pagar

Pero me voy a graduar.  
Mañana mi diploma y título he de  
pagar

Llegó mi momento de despedirte  
Tras años en los que has estado mi  
lado.

Siempre me has ayudado  
Cuando por exámenes me he  
agobiado

Oh Wuolah wuolilah  
Tu que eres tan bonita

A continuación, se va a detallar las tareas utilizadas para esta práctica.

- **Task(Avance):** en esta tarea únicamente se establece la velocidad del motor conectado al puerto b y al puerto c del robot. Esta tendrá el valor de b y c respectivamente. Dicha tarea, como su nombre indica, tiene la función de hacer que el robot avance.

```
22 /*-----  
23 /* Task information:  
24 /* -----  
25 /* Name      : Avance  
26 /* Priority: 1  
27 /* Typ       : EXTENDED TASK  
28 /* Schedule: NON  
29 /* Objective: Calibrate servomotors function  
30 /*-----  
31 TASK(Avance)  
32 {  
33  
34  
35     nxt_motor_set_speed(NXT_PORT_B, b, 1);  
36     nxt_motor_set_speed(NXT_PORT_C, c, 1);  
37  
38  
39     TerminateTask();  
40  
41 }
```

- **Task(Calibra):** esta tarea ha sido creada para corregir el desvío del robot, e ir administrando la potencia adecuada a cada motor para que el robot avance recto.

En primer lugar, se le asigna a **rpmb** las *revoluciones* del motor b. Ésto se hace a través de la función `nxt_motor_get_count()`. Se sigue el mismo procedimiento para rpmb.

A continuación, se comprueba si rpmb es menor que rpmb. En tal caso se comprueba si b es menor que 100. Si es menor se incrementa b, si no, se incrementa c. Con ésto conseguimos que si b tiene menos potencia que c, b se

---

incremente. Si b estuviese ya a máxima potencia se debería disminuir c para así equilibrar ambas potencias. Una vez hecho esto se le asignan la nueva velocidad a los motores.

En caso de que *rpmc* sea menor se debe seguir el mismo procedimiento pero a la inversa. Es decir, se comprueba si *c* es menor que 100. Si es menor se incrementa *c*, si no, se incrementa *b*.

Por último, se resetea el contador de revoluciones. Para ello se usa la función *nxt\_motor\_set\_count()*.

Esta función se ha descrito detalladamente porque será clave para el resto de prácticas.

```
43 /*-----
44 /* Task information:
45 /* -----
46 /* Name      : Calibra
47 /* Priority: 1
48 /* Typ       : EXTENDED TASK
49 /* Schedule: NON
50 /* Objective: Calibrate servomotors function
51 /*-----
52 TASK(Calibra)
53 {
54
55     rpmb=nxt_motor_get_count(NXT_PORT_B);
56     rpmc=nxt_motor_get_count(NXT_PORT_C);
57
58     if(rpmb < rpmc){
59         if(b < 100){
60             b++;
61         }else{
62             c--;
63         }
64         nxt_motor_set_speed(NXT_PORT_B, b, 1);
65         nxt_motor_set_speed(NXT_PORT_C, c, 1);
66     }
67     else if(rpmb > rpmc){
68         if(c < 100){
69             c++;
70         }else{
71             b--;
72         }
73         nxt_motor_set_speed(NXT_PORT_B, b, 1);
74         nxt_motor_set_speed(NXT_PORT_C, c, 1);
75     }
76
77     nxt_motor_set_count(NXT_PORT_B, 0);
78     nxt_motor_set_count(NXT_PORT_C, 0);
79
80
81
82     TerminateTask();
83
84 }
```



- 
- **Task(Final):** Esta función pone los motores a 0 y apaga el sistema.

```
86 /*-----  
87 /* Task information:  
88 /* -----  
89 /* Name      : Final  
90 /* Priority: 2  
91 /* Typ       : EXTENDED TASK  
92 /* Schedule: NON  
93 /* Objective: Stop servomotors  
94 /*-----  
95 TASK(Final)  
96 {  
97  
98     nxt_motor_set_speed(NXT_PORT_B, 0, 1);  
99     nxt_motor_set_speed(NXT_PORT_C, 0, 1);  
100  
101     ShutdownOS(0);  
102  
103  
104     TerminateTask();  
105  
106 }
```

---

## Fichero.oil

En primer lugar, se inicia el contador el cual se iniciará en el ciclo 1 y tendrá un valor máximo de 7000 ns, es decir, 7 s. Usando este contador se crean dos alarmas, Alarma1 y Alarma2.

```
1 #include "kernel.h"
2 #include "kernel_id.h"
3 #include "ecrobot_interface.h"
4
5 /*-----
6  * OSEK declarations
7  *-----
8  DeclareCounter(Contador);
9  DeclareTask(Avance);
10 DeclareTask(Calibra);
11 DeclareTask(Final);
12 int b=100, c=100;
13 int rpmb, rpmc;
14
15 /*-----
16  * Function to be invoked from a category 2 interrupt
17  *-----
18 void user_lms_isr_type2(){
19     SignalCounter(Contador);
20 }
```

La Alarma1 se activará cuando el contador llegue a su valor máximo. Esta alarma activa la tarea final. Por otra parte, la Alarma2 se activará por primera vez a los 25ns y reactivará cada 25ns. Dicha alarma activa la tarea Calibra.

```
27 ALARM Alarma1
28 {
29     COUNTER    = Contador;
30     ACTION     = ACTIVATETASK {
31         TASK    = Final;
32     };
33     AUTOSTART  = TRUE {
34         ALARMTIME    = 7000;
35         CYCLETIME    = 250;
36         APPMODE      = sample_appmodel;
37     };
38 };
```

```
40 ALARM Alarma2
41 {
42     COUNTER    = Contador;
43     ACTION     = ACTIVATETASK {
44         TASK    = Calibra;
45     };
46     AUTOSTART  = TRUE {
47         ALARMTIME    = 25;
48         CYCLETIME    = 25;
49         APPMODE      = sample_appmodel;
50     };
51 };
```

A continuación, se describirán las tareas:

Que no te escriban poemas de amor  
cuando terminen la carrera ▶▶▶▶▶▶▶▶▶▶



WUOLAH

(a nosotros por suerte nos pasa)

No si antes decirte  
Lo mucho que te voy a recordar

Pero me voy a graduar.  
Mañana mi diploma y título he de  
pagar

Llegó mi momento de despedirte  
Tras años en los que has estado mi  
lado.

Siempre me has ayudado  
Cuando por exámenes me he  
agobiado

Oh Wuolah wuolilah  
Tu que eres tan bonita

- **Task(Avance):** en esta tarea el autostart estará igualado a true, por lo tanto, se activará automáticamente la primera. Tendrá prioridad 1, que es la máxima.

```
53 TASK Avance
54 {
55     AUTOSTART = TRUE
56     {
57         APPMODE = sample_appmodel;
58     };
59     PRIORITY = 1;
60     ACTIVATION = 1;
61     SCHEDULE = FULL;
62     STACKSIZE = 512;
63 };
```

- **Task(Calibra):** en este caso, el autostart será false y su prioridad será 3. Dicha tarea tiene menos prioridad que Final porque si no, nunca se activaría.

```
65 TASK Calibra
66 {
67     AUTOSTART = FALSE;
68     PRIORITY = 3;
69     ACTIVATION = 1;
70     SCHEDULE = FULL;
71     STACKSIZE = 512;
72 };
73
```

- **Task(Final):** el autostart está inicializado a false, mientras que su prioridad es 2.

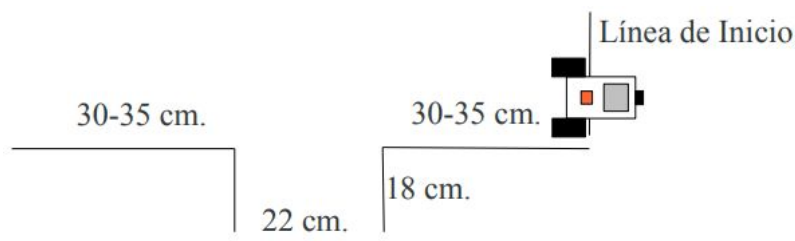
```
74 TASK Final
75 {
76     AUTOSTART = FALSE;
77     PRIORITY = 2;
78     ACTIVATION = 1;
79     SCHEDULE = FULL;
80     STACKSIZE = 512;
81 };
```

Enlace: <https://youtu.be/nNqEdRN5Qdc>

---

## Apartado b

Programar una maniobra de aparcamiento del robot sin tener en cuenta sensores. Se muestra una descripción del entorno de aparcamiento. El robot se posicionará con las ruedas motrices rozando la línea lateral y con las ruedas justo por delante de la línea de inicio.



### Fichero.c

En primer lugar se declaran las tareas *Avance*, *GiroIzq*, *GiroDer*, *Principal* y *Final*. Además, se declara un contador y cuatro variables. Estas variables son las siguientes:

- **b:** tipo int, inicializada a 50. Utilizada para la velocidad del motor conectado al puerto b.
- **c:** tipo int, inicializada a 50. Utilizada para la velocidad del motor conectado al puerto c.
- **rpmb:** tipo int, sin inicializar. Utilizada para contar la vueltas del motor conectado al puerto b.
- **rpmc:** tipo int, sin inicializar. Utilizada para contar la vueltas del motor conectado al puerto c.

---

```

2  #include "kernel.h"
3  #include "kernel_id.h"
4  #include "ecrobot_interface.h"
5
6  /*-----
7  /* OSEK declarations
8  /*-----
9  DeclareTask(Avance);
10 DeclareTask(GiroIzq);
11 DeclareTask(GiroDer);
12 DeclareTask(Final);
13 DeclareTask(Principal);
14 DeclareCounter(Contador);
15
16 /*-----
17 /* Definitions
18 /*-----
19
20 int b=50, c=50;
21 int rpmb, rpmc;
22
23 /*-----
24 /* Function to be invoked from a category 2 interrupt
25 /*-----
26 void user_lms_isr_type2() {
27     SignalCounter(Contador);
28 }
29

```

A continuación, se va a detallar las tareas utilizadas para esta práctica.

- **Task(Avance):** en esta tarea se declara un timeout de 1400 ns que determinará el tiempo que estará activa. Al igual que en la anterior práctica, esta tarea únicamente se establece la velocidad del motor conectado al puerto b y al puerto c del robot. Esta tendrá el valor de b y c respectivamente. Dicha tarea, como su nombre indica, tiene la función de hacer que el robot avance. Antes de terminar llamará al task GiroDer.

```

30 /*-----
31 /* Task information:
32 /* -----
33 /* Name      : Avance
34 /* Priority: ??
35 /* Type      : EXTENDED TASK
36 /* Schedule: ??
37 /* Objective: Activate servomotors forward, taking into account calibration.
38 /*-----
39 TASK(Avance)
40 {
41
42     int timeout = systick_get_ms() + 2400;
43
44     nxt_motor_set_speed(NXT_PORT_B, b, 1);
45     nxt_motor_set_speed(NXT_PORT_C, c, 1);
46
47     while ( systick_get_ms() < timeout);
48     ActivateTask(GiroDer);
49
50     // Terminar la tarea actual
51     TerminateTask();
52 }
53

```

- **Task(GiroDer):** el timeout de esta tarea será de 1300 ns. Se le asigna a **b** el valor 33 y a **c** -10 para poder realizar el giro. Justo antes de terminar el task, se llama a Principal.

```

83 /*-----
84 /* Task information:
85 /* -----
86 /* Name      : GiroDer
87 /* Priority: ??
88 /* Typ       : EXTENDED TASK
89 /* Schedule: ??
90 /* Objective: Activate servomotors to make a right corner, taking into
91 /*            account calibration.
92 /*-----
93 TASK(GiroDer)
94 {
95
96     int time_out = systick_get_ms() + 1300 ;
97
98     b = 33;
99     c = -10;
100     nxt_motor_set_speed(NXT_PORT_B, b, 1);
101     nxt_motor_set_speed(NXT_PORT_C, c, 1);
102
103     while(systick_get_ms() < time_out);
104     ActivateTask(Principal);
105
106     // Terminar la tarea actual
107     TerminateTask();
108 }
109

```

- **Task(Principal):** su timeout será de 325 ns. A continuación, se le asigna de nuevo la velocidad a los motores conectados a los puertos b y c. Posteriormente, se llama a GiroIzq.

```

130 /*-----
131 /* Task information:
132 /* -----
133 /* Name      : Principal
134 /* Priority: ??
135 /* Typ       : EXTENDED TASK
136 /* Schedule: ??
137 /* Objective: Initialize variables and organize tasks.
138 /*-----
139 TASK(Principal)
140 {
141
142     int time_out = systick_get_ms() + 325 ;
143
144     nxt_motor_set_speed(NXT_PORT_B, b, 1);
145     nxt_motor_set_speed(NXT_PORT_C, c, 1);
146
147     while(systick_get_ms() < time_out);
148     ActivateTask(GiroIzq);
149
150     // Terminar la tarea actual
151     TerminateTask();
152 }
153

```



Que no te escriban poemas de amor  
cuando terminen la carrera ▶▶▶▶▶▶▶▶▶▶



WUOLAH

(a nosotros por suerte nos pasa)

No si antes decirte  
Lo mucho que te voy a recordar

Pero me voy a graduar.  
Mañana mi diploma y título he de  
pagar

Llegó mi momento de despedirte  
Tras años en los que has estado mi  
lado.

Siempre me has ayudado  
Cuando por exámenes me he  
agobiado

Oh Wuolah wuolilah  
Tu que eres tan bonita

- **Task(Girolzq):** en dicha tarea se le asignará al timeout un tiempo de 1400 ns. En cuanto a los motores, el valor de b será -12 y el de c 38. Finalmente, se llama a la tarea Final.

```
54 /*-----  
55 /* Task information:  
56 /*-----  
57 /* Name      : Girolzq  
58 /* Priority: ??  
59 /* Typ       : EXTENDED TASK  
60 /* Schedule: ??  
61 /* Objective: Activate servomotors to make a left corner, taking into  
62 /*            account calibration.                                     */  
63 /*-----  
64 TASK(Girolzq)  
65 {  
66  
67     int timeout = systick_get_ms() + 1400 ;  
68  
69     b = -12;  
70     c = 38;  
71  
72     nxt_motor_set_speed(NXT_PORT_B, b, 1);  
73     nxt_motor_set_speed(NXT_PORT_C, c, 1);  
74  
75     while(systick_get_ms() < timeout);  
76     ActivateTask(Final);  
77  
78     // Terminar la tarea actual  
79     TerminateTask();  
80 }  
81
```

- **Task(Final):** Esta función pone los motores a 0 y apaga el sistema.

```
110 /*-----  
111 /* Task information:  
112 /*-----  
113 /* Name      : Final  
114 /* Priority: ??  
115 /* Typ       : EXTENDED TASK  
116 /* Schedule: ??  
117 /* Objective: Stop servomotors  
118 /*-----  
119 TASK(Final)  
120 {  
121  
122     nxt_motor_set_speed(NXT_PORT_B, 0, 1);  
123     nxt_motor_set_speed(NXT_PORT_C, 0, 1);  
124  
125     // Terminar la tarea actual  
126     TerminateTask();  
127  
128 }
```

---

## Fichero.oil

En este caso no habrá alarmas, si no que cada función se activará de manera secuencial gracias a la llamada de una función a otra. Por otra parte, se crea un contador, cuyo valor máximo será 10000.

```
69  COUNTER Contador
70  {
71      MINCYCLE = 1;
72      MAXALLOWEDVALUE = 10000;
73      TICKSPERBASE = 1;
74  };
75  };
```

A continuación, se describirán las tareas:

- **Task(Avance):** en esta tarea el autostart estará igualado a true, por lo tanto, se activará automáticamente la primera. Tendrá prioridad 1, que es la máxima.

```
20  TASK Avance
21  {
22      AUTOSTART = TRUE
23  {
24      APPMODE = sample_appmodel;
25  };
26      PRIORITY = 1;
27      ACTIVATION = 1;
28      SCHEDULE = FULL;
29      STACKSIZE = 512;
30  };
```

- **Task(GiroDer):** en este caso, el autostart será false y su prioridad será 2.

```
41  TASK GiroDer
42  {
43      AUTOSTART = FALSE;
44      PRIORITY = 2;
45      ACTIVATION = 1;
46      SCHEDULE = FULL;
47      STACKSIZE = 512;
48  };
49
```

- 
- **Task(Principal):** el autostart será false y su prioridad será 2.

```
59  TASK Principal
60  {
61      AUTOSTART = FALSE;
62      PRIORITY  = 2;
63      ACTIVATION = 1;
64      SCHEDULE  = FULL;
65      STACKSIZE = 512;
66  };
67
```

- **Task(GiroIzq):** de nuevo, el autostart será false y su prioridad será 2.

```
32  TASK GiroIzq
33  {
34      AUTOSTART = FALSE;
35      PRIORITY  = 2;
36      ACTIVATION = 1;
37      SCHEDULE  = FULL;
38      STACKSIZE = 512;
39  };
40
```

- **Task(Final):** el autostart está inicializado a false, mientras que su prioridad es 4.

```
50  TASK Final
51  {
52      AUTOSTART = FALSE;
53      PRIORITY  = 4;
54      ACTIVATION = 1;
55      SCHEDULE  = FULL;
56      STACKSIZE = 512;
57  };
58
```

Enlace: <https://youtu.be/620yaTg0UUg>

---

## Apartado c

Realizar un movimiento de avance del robot teniendo en cuenta los sensores de ultrasonidos y de pulsación, que estarán colocados en el frontal de avance del robot. El robot avanzará a plena potencia hasta que se encuentre a 1 m. de un obstáculo, que reducirá su potencia a la mitad. A 20 cm. de distancia del obstáculo, el robot reducirá su potencia de avance a un cuarto. El avance continuará hasta que el choque sea detectado por el sensor de pulsación, en cuyo caso, el robot retrocederá durante 1 seg. y girará 90° a la derecha, comenzando de nuevo el mismo procedimiento.

### Fichero.c

En primer lugar se declaran las tareas *Avance*, *Retroceso*, *CambioDir*, *Calibra*, *Principal* y *Muestreo*. Además, se declaran seis variables, las cuales son:

- **b:** tipo int, inicializada a 100. Utilizada para la velocidad del motor conectado al puerto b.
- **c:** tipo int, inicializada a 100. Utilizada para la velocidad del motor conectado al puerto c.
- **sonar1:** tipo int, sin inicializar. Utilizada para tomar el valor de las mediciones realizadas por el sonar.
- **sonar2:** tipo int, sin inicializar. Utilizada para tomar el valor de las mediciones realizadas por el sonar.
- **sonar3:** tipo int, sin inicializar. Utilizada para tomar el valor de las mediciones realizadas por el sonar.
- **pulsador:** tipo int, sin inicializar. Utilizada para saber si el pulsador está activo.

Que no te escriban poemas de amor  
cuando terminen la carrera ▶▶▶▶▶▶▶▶



WUOLAH

(a nosotros por suerte nos pasa)

No si antes decirte  
Lo mucho que te voy a recordar

Pero me voy a graduar.  
Mañana mi diploma y título he de  
pagar

Llegó mi momento de despedirte  
Tras años en los que has estado mi  
lado.

Siempre me has ayudado  
Cuando por exámenes me he  
agobiado

Oh Wuolah wuolilah  
Tu que eres tan bonita

```
1 #include "kernel.h"
2 #include "kernel_id.h"
3 #include "ecrobot_interface.h"
4 #define SONAR_PORT NXT_PORT_S1
5 #define PULSADOR1_PORT NXT_PORT_S2
6
7 /*-----*/
8 /* OSEK declarations */
9 /*-----*/
10
11 //AÑADIR DECLARACION DE TAREAS NUEVAS
12 DeclareTask(Principal);
13 DeclareTask(Avance);
14 DeclareTask(Retroceso);
15 DeclareTask(CambioDir);
16 DeclareTask(Calibra);
17 DeclareTask(Muestreo);
18 int b = 100;
19 int c = 100;
20 int sonar1, sonar2, sonar3, pulsador;
21
22 void ecrobot_device_initialize(){
23     ecrobot_init_sonar_sensor(SONAR_PORT); // Inicializar los sensores activos
24 }
25
26 void ecrobot_device_terminate(){
27     ecrobot_term_sonar_sensor(SONAR_PORT); // Finalizar los sensores activos
28 }
29
30 /*-----*/
31 /* Definitions */
32 /*-----*/
33
34 int sonar;
35
36 /*-----*/
37 /* Function to be invoked from a category 2 interrupt */
38 /*-----*/
39 void user_lms_isr_type2(){}
40
```

A continuación, se va a detallar las tareas utilizadas para esta práctica.

- **Task(Avance):** en primer lugar, se llama a la tarea Muestreo, necesaria para inicializar las variables a un valor. A partir de aquí vienen una serie de comprobaciones. En primer lugar se comprueba si el pulsador está a 1. En caso de que el pulsador esté activado, se llamará a la función Retroceso y CambioDir.

Si el pulsador no está activo, se pasan a comprobar las distancias dadas por el sonar. Si la distancia es menor o igual que 40 la velocidad se reducirá un quinto. Si la distancia es menor o igual que 100 la velocidad se reducirá a la mitad. En caso de que no sea ni menor o igual que 40 ni menor o igual que 100, el robot avanzará con velocidad normal.

---

Por último, una vez establecida la velocidad del robot, se llamará a la función calibra.

```
41 /*-----*/
42 /* Task information: */
43 /* ----- */
44 /* Name      : Avance */
45 /* Priority: ?? */
46 /* Typ      : EXTENDED TASK */
47 /* Schedule: ?? */
48 /* Objective: Activate servomotors forward, taking into account calibration. */
49 /*-----*/
50 TASK(Avance)
51 {
52     ActivateTask(Muestreo);
53
54     if(pulsador==1) {
55         ActivateTask(Retroceso);
56         ActivateTask(CambioDir);
57     }
58     else if(sonar<=40){
59         nxt_motor_set_speed(NXT_PORT_B, b/5, 1);
60         nxt_motor_set_speed(NXT_PORT_C, c/5, 1);
61     }
62     else if(sonar<=100){
63         nxt_motor_set_speed(NXT_PORT_B, b/2, 1);
64         nxt_motor_set_speed(NXT_PORT_C, c/2, 1);
65     }
66     else{
67         nxt_motor_set_speed(NXT_PORT_B, b, 1);
68         nxt_motor_set_speed(NXT_PORT_C, c, 1);
69     }
70
71     ActivateTask(Calibra);
72
73     TerminateTask();
74 }
75
76
77
78
```

- **Task(Calibra):** Se usa la misma función que en las anteriores prácticas. Ya descrita anteriormente.

---

```

80 TASK(Calibra)
81 {
82     int rpmB = nxt_motor_get_count(NXT_PORT_B);
83     int rpmC = nxt_motor_get_count(NXT_PORT_C);
84
85     if ( rpmB < rpmC ){
86         if (b < 100){
87             b++;
88         }else{
89             c--;
90         }
91     }
92     nxt_motor_set_speed(NXT_PORT_B, b,1);
93     nxt_motor_set_speed(NXT_PORT_C, c,1);
94
95 }else if (rpmB > rpmC){
96     if (c < 100){
97         c++;
98     }else{
99         b--;
100     }
101 }
102     nxt_motor_set_speed(NXT_PORT_B, b,1);
103     nxt_motor_set_speed(NXT_PORT_C, c,1);
104
105 }
106
107     nxt_motor_set_count(NXT_PORT_B, 0);
108     nxt_motor_set_count(NXT_PORT_C, 0);
109
110     TerminateTask();
111
112 }

```

- **Task(Muestreo):** En esta función se le asigna el valor de la distancia obtenida por el sonar. Para obtener unas medidas más exactas se realizan tres mediciones y se le asigna el valor medio de las tres. Por último se establece si el pulsador está activado o no.

```

114 TASK(Muestreo)
115 {
116     sonar1=ecrobot_get_sonar_sensor(SONAR_PORT);
117     systick_wait_ms(30);
118     sonar2=ecrobot_get_sonar_sensor(SONAR_PORT);
119     systick_wait_ms(30);
120     sonar3=ecrobot_get_sonar_sensor(SONAR_PORT);
121     systick_wait_ms(30);
122     pulsador=ecrobot_get_touch_sensor(PULSADOR1_PORT);
123     sonar=(sonar1+sonar2+sonar3)/3;
124
125
126     TerminateTask();
127 }
128

```



- 
- **Task(Retroceso):** Se activa un timeout de 1000 ns y se le asigna al motor una velocidad negativa.

```
139 TASK(Retroceso)
140 {
141
142     int timeout=1000;
143
144     nxt_motor_set_speed(NXT_PORT_B, -57, 1);
145     nxt_motor_set_speed(NXT_PORT_C, -57, 1);
146
147     systick_wait_ms(timeout);
148
149
150     TerminateTask();
151 }
```

- **Task(CambioDir):** Se activa un timeout de 1000 ns y se le asigna la velocidad correspondiente al motor, para hacer que el robot gire.

```
162 TASK(CambioDir)
163 {
164     int timeout=1000;
165
166     nxt_motor_set_speed(NXT_PORT_C, 60 , 1);
167     nxt_motor_set_speed(NXT_PORT_B, -10, 1);
168
169     systick_wait_ms(timeout);
170
171     TerminateTask();
172 }
```

- **Task(Principal):** Se crea un bucle infinito donde se llama a la tarea Avance.

```
183 TASK(Principal)
184 {
185     while(1) {
186         ActivateTask(Avance);
187     }
188
189
190     TerminateTask();
191 }
```

Que no te escriban poemas de amor  
cuando terminen la carrera ▶▶▶▶▶▶▶▶



WUOLAH

(a nosotros por suerte nos pasa)

No si antes decirte  
Mañana me voy a recordar  
Lo mucho que te voy a recordar

Pero me voy a graduar.  
Mañana mi diploma y título he de  
pagar

Llegó mi momento de despedirte  
Tras años en los que has estado mi  
lado.

Siempre me has ayudado  
Cuando por exámenes me he  
agobiado

Oh Wuolah wuolilah  
Tu que eres tan bonita

## Fichero.oil

En esta práctica no hay alarmas tampoco si no que cada función se activará de manera secuencial gracias a la llamada de una función a otra. A continuación, se describirán las tareas:

```
1 #include "implementation.oil"
2
3 CPU ATMEL_AT91SAM7S256
4 {
5     OS LEJOS_OSEK
6     {
7         STATUS = EXTENDED;
8         STARTUPHOOK = FALSE;
9         ERRORHOOK = FALSE;
10        SHUTDOWNHOOK = FALSE;
11        PRETASKHOOK = FALSE;
12        POSTTASKHOOK = FALSE;
13        USEGETSERVICEID = FALSE;
14        USEPARAMETERACCESS = FALSE;
15        USERESSCHEDULER = FALSE;
16    };
17
18    APPMODE sample_appmodel{};
19 }
```

- **Task(Principal):** en esta tarea el autostart estará igualado a true, por lo tanto, se activará automáticamente la primera. Tendrá prioridad 1, que es la máxima.

```
65 TASK Principal
66 {
67     AUTOSTART = TRUE
68     {
69         APPMODE = sample_appmodel;
70     };
71     PRIORITY = 1;
72     ACTIVATION = 1;
73     SCHEDULE = FULL;
74     STACKSIZE = 512;
75 };
```

- 
- **Task(CambioDir):** en este caso, el autostart será false y su prioridad será 3.

```
56 TASK CambioDir
57 {
58     AUTOSTART = FALSE;
59     PRIORITY = 3;
60     ACTIVATION = 1;
61     SCHEDULE = FULL;
62     STACKSIZE = 512;
63 };
```

- **Task(Avance):** en esta tarea el autostart será false y su prioridad será 2.

```
29 TASK Avance
30 {
31     AUTOSTART = FALSE;
32     PRIORITY = 2;
33     ACTIVATION = 1;
34     SCHEDULE = FULL;
35     STACKSIZE = 512;
36 };
37
```

- **Task(Calibra):** en dicha task el autostart será false y su prioridad será 5.

```
47 TASK Calibra
48 {
49     AUTOSTART = FALSE;
50     PRIORITY = 5;
51     ACTIVATION = 1;
52     SCHEDULE = FULL;
53     STACKSIZE = 512;
54 };
55
```

- **Task(Muestreo):** de nuevo, el autostart será false y su prioridad será 4.

```
38 TASK Muestreo
39 {
40     AUTOSTART = FALSE;
41     PRIORITY = 4;
42     ACTIVATION = 1;
43     SCHEDULE = FULL;
44     STACKSIZE = 512;
45 };
46
```

- 
- **Task(Retroceso):** el autostart está inicializado a false, mientras que su prioridad es 4.

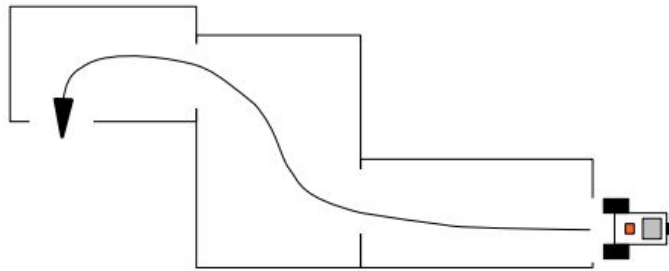
```
20     TASK Retroceso
21     {
22         AUTOSTART    = FALSE;
23         PRIORITY     = 4;
24         ACTIVATION   = 1;
25         SCHEDULE     = FULL;
26         STACKSIZE    = 512;
27     };
```

Enlace: <https://youtu.be/xElaA401SQ0>

---

## Práctica 2.- Manejo básico de Lego Mindstorms NXT: Tareas

Construir y programar un robot con un sensor de ultrasonidos que sea capaz de encontrar la salida de un "laberinto" (Dicho laberinto será una concatenación de pequeñas estancias rectangulares con una única entrada y una única salida). Se incorporará un sensor de pulsación en la parte superior para indicarle al robot que ha salido del laberinto.



Fichero.c

En primer lugar se declaran las tareas *Avance*, *Retroceso*, *CambioDir*, *Calibra*, *Principal* y *Muestreo*. Además, se declaran seis variables, las cuales son:

- **b:** tipo int, inicializada a 100. Utilizada para la velocidad del motor conectado al puerto b.
- **c:** tipo int, inicializada a 100. Utilizada para la velocidad del motor conectado al puerto c.
- **sonar1:** tipo int, sin inicializar. Utilizada para tomar el valor de las mediciones realizadas por el sonar.
- **sonar2:** tipo int, sin inicializar. Utilizada para tomar el valor de las mediciones realizadas por el sonar.
- **sonar3:** tipo int, sin inicializar. Utilizada para tomar el valor de las mediciones realizadas por el sonar.

# La escuela de Ciberseguridad más grande del mundo.

La formación más completa y transversal que demanda el mercado.

Sabemos que es difícil definir tu futuro profesional ¿Te ayudamos?

```
1 #include "kernel.h"
2 #include "kernel_id.h"
3 #include "ecrobot_interface.h"
4 #define SONAR_PORT NXT_PORT_S1
5 #define PULSADOR1_PORT NXT_PORT_S2
6
7 /*-----*/
8 /* OSEK declarations */
9 /*-----*/
10
11 //AÑADIR DECLARACION DE TAREAS NUEVAS
12 DeclareTask(Principal);
13 DeclareTask(Avance);
14 DeclareTask(Retroceso);
15 DeclareTask(CambioDir);
16 DeclareTask(Calibra);
17 DeclareTask(Muestreo);
18 int b = 100;
19 int c = 100;
20 int sonar1, sonar2, sonar3, pulsador;
21
22 void ecrobot_device_initialize(){
23     ecrobot_init_sonar_sensor(SONAR_PORT); // Inicializar los sensores activos
24 }
25
26 void ecrobot_device_terminate(){
27     ecrobot_term_sonar_sensor(SONAR_PORT); // Finalizar los sensores activos
28 }
29
30 /*-----*/
31 /* Definitions */
32 /*-----*/
33
34 int sonar;
35
36 /*-----*/
37 /* Function to be invoked from a category 2 interrupt */
38 /*-----*/
39 void user_lms_isr_type2() {}
```

A continuación, se va a detallar las tareas utilizadas para esta práctica.

- **Task(Avance):** en primer lugar, se llama a la tarea Muestreo, necesaria para inicializar las variables a un valor. A partir de aquí vienen una serie de comprobaciones.

Se comprueban las distancias dadas por el sonar. Si la distancia es menor o igual que 25 la velocidad se reducirá un quinto y se activaran las tareas Retroceso y CambioDir. Si la distancia es menor o igual que 100 la velocidad se reducirá a la mitad. En caso de que no sea ni menor o igual que 40 ni menor o igual que 100, el robot avanzará con velocidad normal.





Por último, una vez establecida la velocidad del robot, se llamará a la función calibra.

```
50 TASK(Avance)
51 {
52     ActivateTask(Muestreo);
53
54     if(pulsador==1) {
55         ActivateTask(Retroceso);
56         ActivateTask(CambioDir);
57     }
58     else if(sonar<=25){
59         nxt_motor_set_speed(NXT_PORT_B, b/5, 1);
60         nxt_motor_set_speed(NXT_PORT_C, c/5, 1);
61         ActivateTask(Retroceso);
62         ActivateTask(CambioDir);
63     }
64
65     else if(sonar<=100){
66         nxt_motor_set_speed(NXT_PORT_B, b/2, 1);
67         nxt_motor_set_speed(NXT_PORT_C, c/2, 1);
68     }
69
70     else{
71         nxt_motor_set_speed(NXT_PORT_B, b, 1);
72         nxt_motor_set_speed(NXT_PORT_C, c, 1);
73     }
74
75     ActivateTask(Calibra);
76
77     // Terminar la tarea actual
78     TerminateTask();
79 }
80 }
```

- **Task(Calibra):** Se usa la misma función que en las anteriores prácticas. Ya descrita anteriormente.

```
82 TASK(Calibra)
83 {
84     int rpmB = nxt_motor_get_count(NXT_PORT_B);
85     int rpmC = nxt_motor_get_count(NXT_PORT_C);
86
87
88     if ( rpmB < rpmC ){
89         if (b < 100){
90             b++;
91         }else{
92             c--;
93         }
94         nxt_motor_set_speed(NXT_PORT_B, b,1);
95         nxt_motor_set_speed(NXT_PORT_C, c,1);
96
97     }else if (rpmB > rpmC){
98         if (c < 100){
99             c++;
100         }else{
101             b--;
102         }
103         nxt_motor_set_speed(NXT_PORT_B, b,1);
104         nxt_motor_set_speed(NXT_PORT_C, c,1);
105
106     }
107
108
109     nxt_motor_set_count(NXT_PORT_B, 0);
110     nxt_motor_set_count(NXT_PORT_C, 0);
111
112     TerminateTask();
113 }
114 }
```



- 
- **Task(Muestreo):** En esta función se le asigna el valor de la distancia obtenida por el sonar. Para obtener unas medidas más exactas se realizan tres mediciones y se le asigna el valor medio de las tres. Por último se establece si el pulsador está activado o no.

```
116 TASK(Muestreo)
117 {
118     sonar1=ecrobot_get_sonar_sensor(SONAR_PORT);
119     systick_wait_ms(30);
120     sonar2=ecrobot_get_sonar_sensor(SONAR_PORT);
121     systick_wait_ms(30);
122     sonar3=ecrobot_get_sonar_sensor(SONAR_PORT);
123     systick_wait_ms(30);
124     pulsador=ecrobot_get_touch_sensor(PULSADOR1_PORT);
125     sonar=(sonar1+sonar2+sonar3)/3;
126
127     // Terminar la tarea actual
128     TerminateTask();
129 }
```

- **Task(Retroceso):** Se activa un timeout de 1000 ns y se le asigna al motor una velocidad negativa.

```
141 TASK(Retroceso)
142 {
143
144     int timeout=1000;
145
146
147     nxt_motor_set_speed(NXT_PORT_B, -27, 1);
148     nxt_motor_set_speed(NXT_PORT_C, -27, 1);
149
150
151     systick_wait_ms(timeout);
152
153     // Terminar la tarea actual
154     TerminateTask();
155 }
```

- 
- **Task(CambioDir):** Se activa un timeout de 1000 ns y se le asigna la velocidad correspondiente al motor, para hacer que el robot gire.

```
166 TASK(CambioDir)
167 {
168
169     int timeout=1000;
170
171
172     nxt_motor_set_speed(NXT_PORT_C, 30 , 1);
173     nxt_motor_set_speed(NXT_PORT_B, -10, 1);
174
175
176     systick_wait_ms(timeout);
177
178     // Terminar la tarea actual
179     TerminateTask();
180 }
```

- **Task(Principal):** Se crea un bucle infinito donde se llama a la tarea Avance.

```
191 TASK(Principal)
192 {
193     while(1) {
194         ActivateTask(Avance);
195     }
196
197     // Terminar la tarea actual
198     TerminateTask();
199 }
200
```

Que no te escriban poemas de amor  
cuando terminen la carrera ▶▶▶▶▶▶▶▶



WUOLAH

(a nosotros por suerte nos pasa)

No si antes decirte  
Lo mucho que te voy a recordar

Pero me voy a graduar.  
Mañana mi diploma y título he de  
pagar

Llegó mi momento de despedirte  
Tras años en los que has estado mi  
lado.

Siempre me has ayudado  
Cuando por exámenes me he  
agobiado

Oh Wuolah wuolilah  
Tu que eres tan bonita

## Fichero.oil

En esta práctica no hay alarmas tampoco si no que cada función se activará de manera secuencial gracias a la llamada de una función a otra. A continuación, se describirán las tareas:

```
1 #include "implementation.oil"
2
3 CPU ATMEL_AT91SAM7S256
4 {
5     OS LEJOS_OSEK
6     {
7         STATUS = EXTENDED;
8         STARTUPHOOK = FALSE;
9         ERRORHOOK = FALSE;
10        SHUTDOWNHOOK = FALSE;
11        PRETASKHOOK = FALSE;
12        POSTTASKHOOK = FALSE;
13        USEGETSERVICEID = FALSE;
14        USEPARAMETERACCESS = FALSE;
15        USERESSCHEDULER = FALSE;
16    };
17
18    APPMODE sample_appmodel{};
19 }
```

- **Task(Principal):** en esta tarea el autostart estará igualado a true, por lo tanto, se activará automáticamente la primera. Tendrá prioridad 1, que es la máxima.

```
65 TASK Principal
66 {
67     AUTOSTART = TRUE
68     {
69         APPMODE = sample_appmodel;
70     };
71     PRIORITY = 1;
72     ACTIVATION = 1;
73     SCHEDULE = FULL;
74     STACKSIZE = 512;
75 };
```

- **Task(CambioDir):** en este caso, el autostart será false y su prioridad será 3.

```
56 TASK CambioDir
57 {
58     AUTOSTART = FALSE;
59     PRIORITY = 3;
60     ACTIVATION = 1;
61     SCHEDULE = FULL;
62     STACKSIZE = 512;
63 };
64
```

- 
- **Task(Avance):** en esta tarea el autostart será false y su prioridad será 2.

```
29  TASK Avance
30  {
31      AUTOSTART    = FALSE;
32      PRIORITY     = 2;
33      ACTIVATION   = 1;
34      SCHEDULE     = FULL;
35      STACKSIZE    = 512;
36  };
```

- **Task(Calibra):** en dicha task el autostart será false y su prioridad será 5.

```
47  TASK Calibra
48  {
49      AUTOSTART    = FALSE;
50      PRIORITY     = 5;
51      ACTIVATION   = 1;
52      SCHEDULE     = FULL;
53      STACKSIZE    = 512;
54  };
```

- **Task(Muestreo):** de nuevo, el autostart será false y su prioridad será 4.

```
38  TASK Muestreo
39  {
40      AUTOSTART    = FALSE;
41      PRIORITY     = 4;
42      ACTIVATION   = 1;
43      SCHEDULE     = FULL;
44      STACKSIZE    = 512;
45  };
```

- 
- **Task(Retroceso):** el autostart está inicializado a false, mientras que su prioridad es 4.

```
20 TASK Retroceso
21 {
22     AUTOSTART = FALSE;
23     PRIORITY  = 4;
24     ACTIVATION = 1;
25     SCHEDULE  = FULL;
26     STACKSIZE = 512;
27 };
```

Enlace: <https://youtu.be/EDPwCoeMKm8>

## Práctica 3.- Manejo avanzado de Lego Mindstorms NXT: Tareas y Comunicaciones

Tras intentar realizar el código repetidas veces y probar, no fuimos capaces de realizar correctamente la práctica.